



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第十一章 软件的质量属性

软件质量属性

- **软件质量属性**指软件的非功能需求，能深刻影响软件产品的质量
- 作为系统（软硬件）的质量属性
 - 性能
 - 可用性
 - 安全性
 - 可扩展性
- 软件特有的质量属性
 - 可维护性
 - 可修改性
 - 可测试性
 - 可追踪性
 - 可复用性



可维护性

- 是软件在部署后能 **被更改** 的程度
- 更改的原因
 - 修复残留的错误
 - 解决性能问题
 - 软件需求的变更
- 要求软件为变化和适应性而设计
 - 文档健全并保持更新
 - 在设计中包含设计决策
 - 可修改性、可追踪性、可测试性都为此服务

在很多情况下，
软件可维护性
实际与软件可
演化性等价



可修改性

- 是软件在最初的部署期间及之后能够 **被修改** 的程度
- 提高可修改性的手段：
 - 以定义良好的接口为基础的模块化设计（**信息隐藏**）
 - 在类和构件级别提供信息隐藏
 - 在子系统级别提供关注点分离
 - 具体的设计决策
 - 将每个有限状态机封装在一个单独的状态机类中
 - 将每个连接到单独的外部设备、系统或用户的接口封装到单独的边界类中
 - 将每个单独的数据结构封装在单独的数据抽象类中
 - 基于构件的软件架构设计方法



可测试性

- 是软件能够 **被测试** 的程度
- 提高可修改性的手段：
 - 在软件生存周期早期建立一个软件测试计划
 - 在软件开发的同时，并行地规划测试用例的开发
 - 具体的策略
 - 在需求阶段，通过用例模型开发功能性测试用例，需要为每个用例场景开发一个测试用例
 - 在架构设计阶段，开发集成测试用例，用于测试相互通信的构件间的接口，通过交互图中的消息序列分析
 - 在构件内部算法详设和编码阶段，开发单元测试用例来测试独立的构件（类）



可追踪性

- 是产品的每个阶段能被追踪到之前的阶段的程度
- 需求的可追踪性用于确保每个软件需求都被设计和实现了
 - 需求被**描述**为用例
 - 用例被**细化**到一个基于用例的交互图
 - 交互图被**集成**为软件架构设计

形成了从需求到软件设计与实现的追踪关系



可扩展性

- 是系统在其最初部署之后能够生长的程度
- 从系统角度看
 - 硬件方面，分布式系统可通过添加结点来扩展
 - 软件方面，基于构件的分布式软件架构比集中式设计更具有扩展性



可复用性

- 是软件能够被 **复用** 的程度
- 不同层次的复用
 - 以类为单位
 - 以构件为单位
 - 以构件及其相互连接构成的整个设计或子系统为单位——体系结构的复用
 - 软件产品线方法



性能

- 在设计时对系统进行性能建模对确定系统是否满足其性能目标非常重要
- 性能包括吞吐量、响应时间、承载能力等
- 性能建模方法
 - 队列建模
 - 仿真建模



安全性

- 软件潜在的威胁有

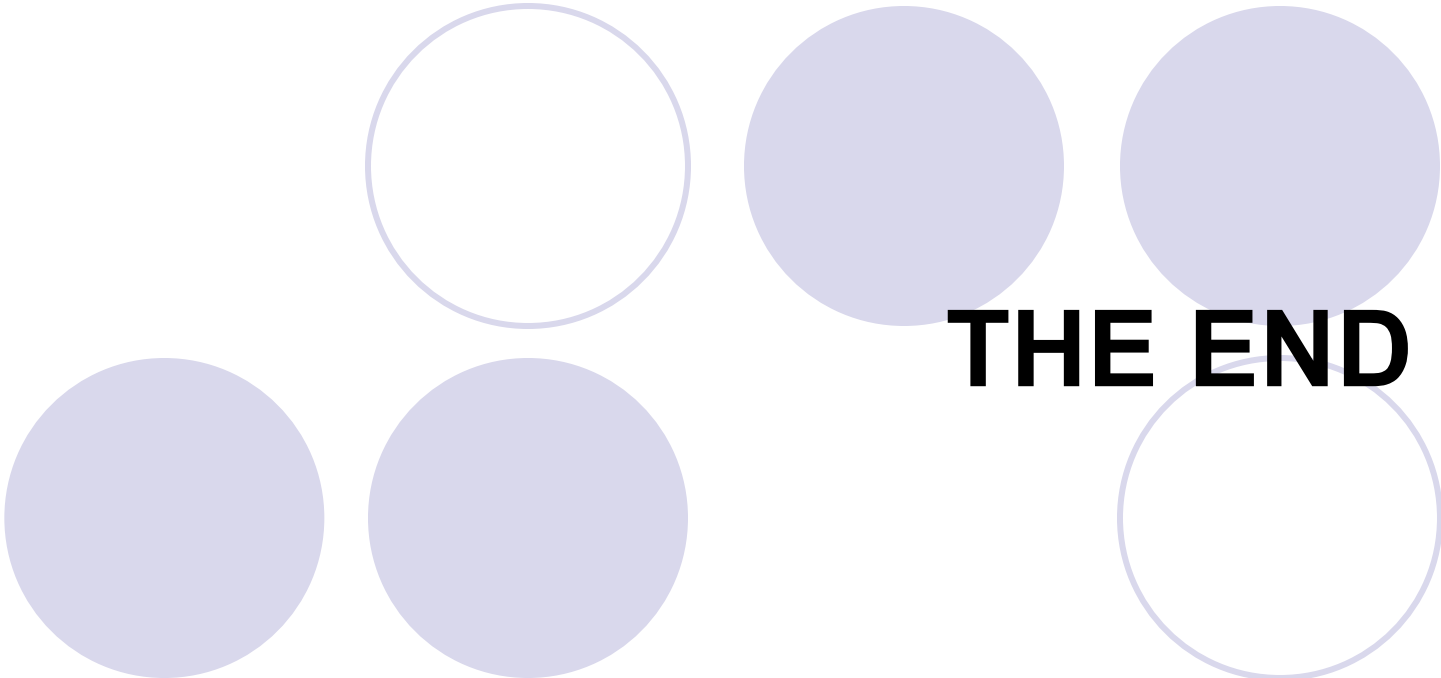
- 系统穿透
- 违反授权
- 保密信息披露
- 完整性破坏
- 抵赖
- 拒绝服务



可用性

- 系统失效及其对用户和其他系统的影响
- 可用性的类型
 - 可预期的——主观安排，如系统预定的维护
 - 不可预期——计划外的失效
- 保持可用性的方法
 - 恢复机制：热备，防止硬件单点失效
 - 基于构件的分布式软件架构：部署到带有分布式控制、数据和服务的多结点上，防止软件单点失效



Five light purple circles are arranged in a pentagonal pattern around the text. Two circles are solid, and three are hollow outlines.

THE END